

OPTIMIZING SINGLE CORE INTERVAL-MASKED TOP- K PEARSON CORRELATION

Oliver Bergqvist, Andres Navaro Pedregal, Luhao Liu (Team 41)

Department of Computer Science
ETH Zurich, Switzerland

ABSTRACT

We optimize an Interval-Masked Streaming Top- K Pearson Correlation algorithm for a single core. In order to guarantee an "apples-to-apples" comparison in terms of FLOP-counts, we start from a logical baseline, that already exploits correlation matrix symmetry and a mathematical-equivalent-optimized variance formula. We then apply a structured sequence of optimizations: instruction-level parallelism, explicit AVX2/FMA vectorization, 4×2 register blocking, and a feature reordering that turns the interval structure into early loop termination and contiguous memory access. On a Kaby Lake core, the final implementation reaches 10.4 flops/cycle (33% of AVX2+FMA peak performance), a $53\times$ speedup over the logical baseline when features span 80% of each row and $26\times$ when features only span 30% of each row.

1. INTRODUCTION

Motivation. The computation of pairwise Pearson correlation coefficients [1] across high-dimensional feature sets is a common operation in data analytics, used for example in gene co-expression modeling [2]. As more data arrives as time-oriented streams, static matrix computations are increasingly replaced by dynamically windowed correlation tracking [3]. The Interval-Masked Streaming Top- K Pearson Correlation algorithm addresses these demands by introducing dynamic filters on the valid dimensional ranges and streaming only the top- K highest values, thereby avoiding the full materialization of the intermediate correlation matrix.

Optimizing this algorithm is harder than it first appears. Tuned libraries such as BLAS reach peak performance only on dense data with uniform access patterns. Interval masking, however, gives each feature pair its own start and end indices. These non-uniform bounds break standard cache-blocking and add control-flow branching that hampers SIMD vectorization, so reaching high throughput requires a custom implementation. Prior work addresses related but different problems. Schubert and Gertz [4] optimize the numerically stable computation of (co-)variance, which we adopt for our preprocessing, but they consider neither the

partially overlapping intervals nor the top- K selection. TOP-COP [5] mines the top- K strongly correlated pairs, but its focus is on pruning the search algorithmically rather than on the single-core, vectorized execution we study here.

Contribution. In this report, we detail a series of software optimizations applied to the Interval-Masked Streaming Top- K Pearson Correlation algorithm executing on a single CPU core. Using explicit vectorization, instruction-level parallelism (ILP), and cache-friendly data reordering, our optimized C++ implementation substantially outperforms the straightforward baseline. For a problem size of 2048 features across 512 dimensions, we achieve: (1) a $53\times$ speedup when 80% of the values fall within the valid interval, and (2) a $26\times$ speedup when only 30% of the values are valid, both measured against the logical baseline.

2. BACKGROUND ON THE ALGORITHM

Algorithm. The Interval-Masked Streaming Top- K Pearson Correlation algorithm is designed to identify the K most strongly correlated features for every feature within a high-dimensional dataset, constrained by feature-specific valid intervals. Given an input data matrix X of size $F \times S$ (where F is the number of features and S is the number of dimensions/samples), along with two arrays, start and end, of length F that define the valid interval $[\text{start}[f], \text{end}[f])$ for each feature, the algorithm executes in three distinct phases:

Phase 1: Preprocessing. For each feature $f \in F$, the algorithm isolates the valid data window to compute the local mean (μ_f) and sample standard deviation (σ_f). Let $S_f = \text{start}[f]$, $E_f = \text{end}[f]$, and the interval length $N_f = E_f - S_f$. The statistics can be shown as:

$$\mu_f = \frac{1}{N_f} \sum_{i=S_f}^{E_f-1} X_{f,i}$$
$$\sigma_f = \sqrt{\frac{1}{N_f - 1} \sum_{i=S_f}^{E_f-1} (X_{f,i} - \mu_f)^2}$$

In the actual algorithm, the σ_f is calculated after applying the mathematical relation between mean and variance:

$$\sigma_f = \sqrt{\frac{1}{N_f - 1} \left(\sum_{i=S_f}^{E_f-1} X_{f,i}^2 - 2.0 \times \mu_f \sum_{i=S_f}^{E_f-1} X_{f,i} + N_f \mu_f^2 \right)}$$

Further, the z -normalized values, which we will need in Phase 2, are also pre-computed here to avoid repeated calculations.

$$z_{f,i} = \frac{(X_{f,i} - \mu_f)}{\sigma_f}$$

Phase 2: Overlap Correlation Calculation. For every unique pair of features f_0 and f_1 , the algorithm determines the strict overlap of their valid intervals. The bounds of this overlapping region are defined by $L = \max(S_{f_0}, S_{f_1})$ and $R = \min(E_{f_0}, E_{f_1})$.

Assuming a meaningful overlap exists (i.e., $R - L > 1$), let the overlap length be $O = R - L$. The interval-masked Pearson correlation coefficient (ρ_{f_0, f_1}) is then calculated by taking the dot product of the z -normalized values within this specific overlapping window:

$$\rho_{f_0, f_1} = \frac{1}{O - 1} \sum_{i=L}^{R-1} z_{f_0, i} z_{f_1, i}$$

Since $\rho_{f_0, f_1} = \rho_{f_1, f_0}$, only the upper half triangular ($f_1 > f_0$) is actually computed in the algorithm. Note that self correlations are excluded in our setup.

Phase 3: Streaming Top- K Selection. Rather than storing the full $O(F^2)$ correlation matrix, the algorithm streams the calculated coefficients into a bounded selection data structure (such as a min-heap of size K). For each feature f_0 , it maintains only the K indices of f_1 that yield the highest values of ρ_{f_0, f_1} . The final output is an $F \times K$ index matrix representing the top- K correlated peers for every feature.

Cost Analysis. We solely focus on the floating point arithmetic for cost analysis. In Phase 1, for each feature f with valid values N_f , N_f FLOPs are needed to accumulate the values, $2N_f$ FLOPs to accumulate the square of values and $2N_f$ FLOPs for normalization. An additional 10 FLOPs are also necessary for the calculation of μ_f and σ_f . In Phase 2, for each valid pair (f_0, f_1) where $f_1 > f_0$ with O_{f_0, f_1} overlapping values, $2O_{f_0, f_1}$ FLOPs are needed to accumulate the multiplications and 1 additional FLOP to perform the division. In Phase 3, no FLOPs are counted as we do not perform any arithmetic computation directly.

In total, we have a total cost in FLOPs of

$$\sum_{f \in F} (5N_f + 10) + \sum_{(f_0, f_1) \in (F, F), f_1 > f_0} (2O_{f_0, f_1} + 1)$$

In terms of data transfer, we only count compulsory misses, which consist of the input matrix, the normalized matrix, the heap insertions, and the final top- K results.

$$(4 \times \sum_{f \in F} 2N_f) + (8 \times F \times K) + (4 \times F \times K) \text{ Bytes}$$

3. OPTIMIZATION PERFORMED

Logical Baseline. The logical baseline is implemented exactly as described in Section 2. This baseline already incorporates several algorithmic reductions: variance formula rewriting, z -score normalization pre-computation, and the exploitation of the correlation matrix’s symmetry. We also implemented a purely naive baseline omitting these techniques, but they change the FLOP count and the underlying cost model. Therefore, all subsequent analysis, profiling, and experiments refer to this logical baseline unless otherwise specified.

To identify the primary computational bottleneck, we conducted a runtime profiling using a problem size of $F = 128$ features and $S = 8192$ dimensions. The profiling revealed that Phases 1, 2, and 3 account for 3.4%, 96.6%, and 0.1% of the total CPU cycles, respectively. This indicates that the Phase 2 inner loop—specifically the calculation of the dot product over the overlapping interval—dominates the execution time in the logical baseline. Consequently, our primary optimizations are targeted at this specific loop:

```
for (uint32_t t = L; t < R; ++t) {
    acc += Z[i*S + t] * Z[j*S + t];
}
corr = acc / (R - L - 1);
```

However, by Amdahl’s Law, as pipelining-aware optimization and vectorization cut Phase 2’s runtime, the formerly negligible Phases 1 and 3 account for a growing fraction of the total runtime. To prevent these phases from emerging as secondary bottlenecks, the memory and vectorization optimizations discussed in the following sections are systematically applied to all phases wherever viable.

ILP Optimization. In the baseline implementation, the inner loop’s accumulator depends directly on the computation from the previous iteration. This loop-carried data dependency creates a strict read-after-write (RAW) hazard. Because this hazard cannot be bypassed via hardware register renaming, it prevents the full utilization of the CPU pipeline and severely limits instruction-level parallelism (ILP). To resolve this bottleneck, we split the reduction across several independent accumulators, which removes the single dependency chain and lets the out-of-order core overlap successive multiply-adds. The loop is unrolled only as the mechanism that exposes these independent accumulators; the unrolling itself is not the source of the speedup. The number of accumulators that is useful is dictated by the microarchitecture’s instruction throughput and latency. On the target Kaby Lake core (Skylake refresh microarchitecture),

a Fused Multiply-Add (FMA) instruction has a latency of 4 cycles and can be issued twice per cycle, so 8 independent accumulators are needed to fully hide the execution latency. In this scalar stage we use 4 accumulators, and reach the full 8 only later with the register-blocked vector kernel.

```
uint32_t t = L;
for (; t + 3 < R; t += 4) {
    acc0 += Z[i*S + t + 0] * Z[j*S + t + 0];
    acc1 += Z[i*S + t + 1] * Z[j*S + t + 1];
    acc2 += Z[i*S + t + 2] * Z[j*S + t + 2];
    acc3 += Z[i*S + t + 3] * Z[j*S + t + 3];
}
for (; t < R; ++t) {
    acc0 += Z[i*S + t] * Z[j*S + t];
}
corr = (acc0 + acc1 + acc2 + acc3) / (R - L - 1);
```

Similar optimizations were also applied to Phase 1. This configuration represents the highest-performing scalar version before the introduction of explicit vectorization.

Vectorization. Next, we applied explicit vectorization to fully exploit the Single Instruction, Multiple Data (SIMD) capabilities of the CPU. Because our normalized data is stored as 32-bit floating-point values, utilizing 256-bit Advanced Vector Extensions (AVX) instructions allows us to execute 8-way parallel computations.

```
uint32_t t = L;
for (; t + 7 < R; t += 8) {
    __m256 v_i = _mm256_loadu_ps(&Z[i*S + t]);
    __m256 v_j = _mm256_loadu_ps(&Z[j*S + t]);
    v_acc = _mm256_fmadd_ps(v_i, v_j, v_acc);
}
// hsum256 returns the sum of values in the vector
float acc = hsum256(v_acc);
for (; t < R; ++t) {
    acc += Z[i*S + t] * Z[j*S + t];
}
corr = acc / (R - L - 1);
```

While vectorization improves throughput, consolidating the inner loop into a single vector accumulator reintroduces a loop-carried data dependency. ILP therefore drops, leaving the execution pipeline partially starved. To resolve this pipeline underutilization and achieve peak performance, we reintroduced the multiple-accumulator strategy through blocking, which is detailed in the subsequent section.

Blocking. For simplicity in our vectorized implementation, we assume that all interval boundaries, as well as the total dimension size S , are strictly divisible by 8. For a fully generalized implementation, this constraint can be easily relaxed by aligning the start and end indices to the nearest multiple of 8, provided that the out-of-bounds normalized values are padded with zeros to ensure they do not affect the final accumulation. Because this corner case has a negligible impact on our primary optimization goals, we opted to ignore it in our implementation.

Combining loop blocking (tiling) with vectorization yields two major architectural benefits: improved cache locality and restored ILP. In Phase 2, the naive double outer loop iterates over feature pairs in a linear order: $(0, 1), (0, 2), \dots, (0, F - 1), (1, 2), (1, 3), \dots$. When F and S become sufficiently large, the cache can no longer house the entire Z matrix. Under these conditions, only the values of the outer-loop feature are retained in the cache, while iterating through the inner loop triggers continuous cache evictions across the entire matrix. To mitigate this, we restructured the computation into 4×2 blocks. The access pattern is thus transformed into block-wise iterations: $([0, 1, 2, 3], [4, 5]), ([0, 1, 2, 3], [6, 7]), \dots, ([0, 1, 2, 3], [F - 2, F - 1]), ([4, 5, 6, 7], [8, 9]), \dots$ (Note that the diagonal components, such as the correlations within the block $[0, 1, 2, 3]$ against itself, are computed via a specialized routine prior to entering the inner loop). By evaluating a 4×2 block of the correlation matrix at once, the data loaded for the 4 outer features is reused across the 2 inner features, reducing memory traffic and the cache miss rate by a factor of four compared to the unblocked version. Processing 8 independent feature pairs per iteration also uses 8 separate vector accumulators, enough to keep both FMA units busy and restore ILP.

The blocked kernel is long and intrinsic-heavy, so we omit the snippet here; the full source is in Section 8. Similar blocking and vectorization strategies were also deployed in Phase 1.

Sorting. An unclustered distribution of valid intervals degrades cache locality and incurs redundant memory accesses. Pre-sorting the features by their starting indices lets us terminate the inner loop early: once a candidate feature’s starting index exceeds the target feature’s ending index, all subsequent features are disjoint and we can exit. As shown in Figure 1, clustering by overlap also produces contiguous strips of accessed memory that fit better in the caches. Iterating over these sorted blocks yields a predictable stride pattern, which improves the hardware prefetcher’s hit rate.

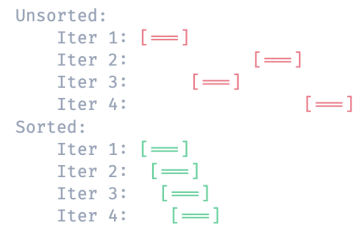


Fig. 1. Different access patterns before and after sorting.

This data reordering mechanism is implemented alongside all previously discussed architectural optimizations. During Phase 1, an initial sorting routine generates an index permutation array (Step 1), and the subsequent normalization routine physically reorders the rows of the Z matrix in

memory according to this permutation (Step 2). In Phase 2, the min-heap insertion logic is updated to dereference the permutation array, ensuring the original feature indices are correctly preserved during the streaming Top- K selection (Step 3). Finally, in Phase 3, the output rows are scattered back into their original locations via the permutation array, ensuring the final output matrix remains consistent with the input data layout (Step 4).

```
// Phase 1
// Step 1: sort by start
for (i) perm[i] = i;
std::sort(perm, perm+F,
    [&](a,b){start[a] < start[b]});
for (si) { // cache sorted bounds
    sorted_starts[si] = starts[perm[si]];
    sorted_ends[si] = ends[perm[si]];
}
// Step 2: normalize, read original row
// perm[si], write sorted row si
Z[si*S + t] = (X[perm[si]*S+t] - mean) * inv_std;
// Phase 2
// Step 3: sorted-space dot-prod: orig. idx → heap
heap_insert(&heaps[i*K], v, perm[j]);
// Phase 3
// Step 4: scatter through perm
heap_to_topK(&heaps[si*K], &topK[perm[si]*K]);
```

The benefit of sorting depends on the density of the overlapping intervals. With sparse intervals, the early-exit condition triggers often and prunes a large share of the evaluated pairs. Conversely, when the intervals are dense and mostly overlap, the computational reduction is small, and the gains come instead from better cache locality and prefetching.

4. EXPERIMENTAL RESULTS

We evaluate the each optimization by sweeping the number of features F and the sample count S separately. All experiments are run in both settings provided, dense (LONG) and sparse (SHORT), because the two stress the algorithm very differently. We answer the following questions: how much each step contributes to the latency, why the two F and S behave differently, how performance scales when scaling F and S , and what limits the best code.

Experimental setup. All experiments are run on a single Kaby Lake core (Intel i7-7700HQ, Skylake-refresh microarchitecture) at a fixed 2.80 GHz, with 32 KB L1-D, 256 KB L2, and a 6 MB shared L3. To make the cycle counts reproducible we disable Turbo Boost, pin the frequency governor to performance, disable the core sibling so that nothing shares the physical core, and pin the benchmark to the core with `taskset`. The code is compiled with g++ 12.2.0. The compiled flags vary by each step and are themselves part of the experiment. The logical baseline is left unoptimized (`-O0`). The two manual-ILP variants share identical source

compiled with `-O3 -march=native -ffast-math` and differ only in whether the auto-vectorizer is turned off or not by `-fno-tree-vectorize`. The explicit-SIMD and sorting variants additionally enable `-mavx -mfma`. The single-precision AVX2+FMA peak on this core is 32 flops/cycle (2 FMA units $\times$ 8 fp32 lanes $\times$ 2 flops).

We use PAPI [6] to count the cycles, instructions, cache misses, and flop counts to be able to understand the bottlenecks. We measure under cold-cache conditions, which we consider representative of real use: before each measured batch we `clflush` the input matrix, the interval bounds, and the output buffer. Each batch is sized to span at least 10^8 cycles and one warm-up batch is discarded, so that timer and PAPI overhead are negligible. We report the median over the batches.

The inputs are generated synthetically by a provided generator. The F -sweep ranges over $F \in \{8, \dots, 2048\}$ at a fixed $S = 512$, and the S -sweep ranges over $S \in \{32, \dots, 16384\}$ at a fixed $F = 512$. The upper ends of both ranges are set by how long the experiments take rather than by the algorithm, because the slowest variant we measure is the unoptimized logical baseline and the total work grows quickly: the number of feature pairs grows as F^2 and the work per pair grows with S . In our runs the full F -sweep over all variants and both regimes already took about six hours, and on the S -sweep a single baseline point cost about four hours at $S = 8192$ and roughly doubled with each doubling of S , so that the complete S -sweep did not finish within 26 hours and had to be stopped. We therefore cap the sweeps at $F = 2048$ and $S = 16384$, which is the largest range that still completes overnight while comfortably covering the transition from data fitting in cache to spilling to main memory. Each feature is given a fixed-length valid subinterval placed at a random offset (aligned to a multiple of 16), and we report the top- K neighbours with $K = 16$. The interval length can be of two types, 80% of the row (LONG) and to 30% (SHORT). Sample values are i.i.d. Gaussian. As the algorithm does not depend on the values of the data, then the performance is the same for different generation. We tested that with the BLOCK-CORR and GAUSSIAN, so we report only the Gaussian numbers. Throughout, the performance on the y -axis is the analytical useful-flop count of Section 2 divided by the measured cycles. We want to be explicit about this choice, since we do record the hardware floating-point counter `PAPI_SP_OPS` as well and deliberately keep it off the axis. Because the SIMD kernels pad the invalid lanes with zeros, `PAPI_SP_OPS` also counts the arithmetic spent on that padding, which at $F = 2048$ inflates the hardware count by up to $4.8\times$ on SHORT (the padding factor in Table 1). Putting that hardware count on the axis would make the dense algorithm look fast on sparse data and make sorting, whose whole purpose is to remove the padding, appear as a slowdown. It

would also break the rule of not comparing variants whose operation counts differ substantially. We therefore use the analytical count as the measure of useful work, and use `PAPI_SP_OPS` separately as a direct measurement of the padding overhead. Because the logical baseline already changes the operation count relative to a purely naive version, that naive version is excluded from the plots and all speedups are stated against the logical baseline.

Per-step gains with the number of features. Figure 2 lets us see each optimization performance and Table 1 gives the PAPI counters behind each line at $F = 2048$ so we can attribute every change to a concrete cause. Unless stated otherwise we quote the dense `LONG` regime at $F = 2048$ to keep it consistent with the table. The first thing we notice is how far the logical baseline sits below every other line, at only 0.20 flops/cycle. The counters tell us why, compiled at `-O0` the baseline executes 13.8 instructions for every useful flop, so even though it sustains a good IPC of 2.7, almost all of that work is loads and memory accesses rather than useful instructions. At this stage the bottleneck is the instruction count, not the hardware.

Manual ILP with four independent accumulators, still scalar, gives the largest relative jump of performance of $6.2\times$, to 1.23 flops/cycle. Two effects combine here. `-O3` strips out the redundant instructions, dropping the instruction-per-flop ratio from 13.8 to 2.7, and the four accumulators break the loop-carried dependency in the Phase-2 reduction, lifting the IPC to 3.3. The baseline’s single accumulator forced each FMA to wait for the previous one to finish. On the other hand, with four partial sums the core always has independent work to issue, which is exactly the instruction-level parallelism we wanted. This is the fastest purely scalar code we obtained.

The third line is the one we found counter-intuitive. Its source is identical to the previous step and the only change is that we stop passing `-fno-tree-vectorize`, that is, we let the compiler auto-vectorize. Instead of improving, performance almost halves, to 0.54 flops/cycle, and the IPC falls to 1.7. The clearest signal is in the resource-stall fraction jumps from 1.6% for the hand-tuned scalar loop to 32% once the compiler vectorizes, so the core now spends roughly a third of its cycles stalled. Reading the generated assembly explains it. GCC does vectorize the loop, but because the interval bounds are not known at compile time it cannot guarantee that an eight-wide load stays inside the valid region, so it surrounds the vector loop with extra scalar code that handles the few elements at the start and end one at a time and adds shuffle instructions to line the data up. The practical lesson we take away is that this kernel will not auto-vectorize well, so the gains have to be engineered by hand. Doing exactly that, explicit `AVX2/FMA` vectorization, achieves a performance of 3.78 flops/cycle ($19\times$). The zero-padding trick is what makes it work. We pad

the invalid entries with zeros which turns the masked dot product into a dense, branch-free eight-wide one, and the instruction-per-flop ratio reduces to 0.51, about one instruction per two useful flops. The bottleneck now is because as we fold everything into one vector accumulator, it reintroduces the loop-carried dependency we had removed, now on the vector FMA, so the IPC is only 1.9.

Multi-accumulator 4×2 register blocking breaks that loop-carried dependency and cuts the memory traffic at the same time. We obtain a performance of 9.19 flops/cycle ($47\times$). Computing a 4×2 block of feature pairs at once gives eight independent FMA chains. Since an FMA on this core has a four-cycle latency and the two FMA units can each start one per cycle, roughly eight need to be in flight to keep both units busy, and eight is exactly what the block provides, so the IPC increases to 2.3. Just as important, each loaded vector is now reused across all the pairs in the block instead of being re-read from memory for every single pair, and the cache counters show this directly: L2 misses drop by $4.2\times$ and L3 misses by $5.5\times$.

Finally, sorting the features by interval start adds another 13% in `LONG`, to 10.43 flops/cycle ($53\times$ overall). Here every pair already overlaps so there is nothing to skip, and the gain comes only from processing features in sorted order, which makes the accesses contiguous and predictable for the hardware prefetcher and removes a further 20% of the L2 misses. Note that this is different for the `SHORT` as explained next.

Why the `LONG` and `SHORT` differ. The right panel of Figure 2 looks quite different, and the comparison is the most instructive one in the report. In `SHORT`, multi-accumulator reaches only 1.83 flops/cycle, barely above explicit vectorization, yet sorting more than doubles it to 4.11 ($26\times$ end-to-end). Here the hardware counter is decisive. Comparing the flop counts from PAPI against the analytical useful-flop count gives a ratio of 4.8 for multi-accumulator in `SHORT`, against only 1.19 in `LONG`. In other words, the dense block always computes the full row length regardless of the real overlap, so on the short `SHORT` intervals it spends almost five flops of hardware work for every flop that matters, most FMA slots going to padded zeros. The early-exit test that sorting enables skips the roughly 40% of pairs that do not overlap at all and trims the padded tails, dropping the padding factor back to 1.08 and accounting for essentially the entire $1.83 \rightarrow 4.11$ improvement. The cache counters confirm the same effect from the memory side, with sorting cutting `SHORT` L2 misses from 16M to 4.3M. In `LONG` the overlaps are dense and the padding overhead not notizable, only 3%. Sorting is therefore the one optimization whose payoff is dictated by the shape of the input rather than by the hardware.

Scaling with the dot-product length. Figure 3 fixes $F = 512$ and grows S , the length of each dot product. The

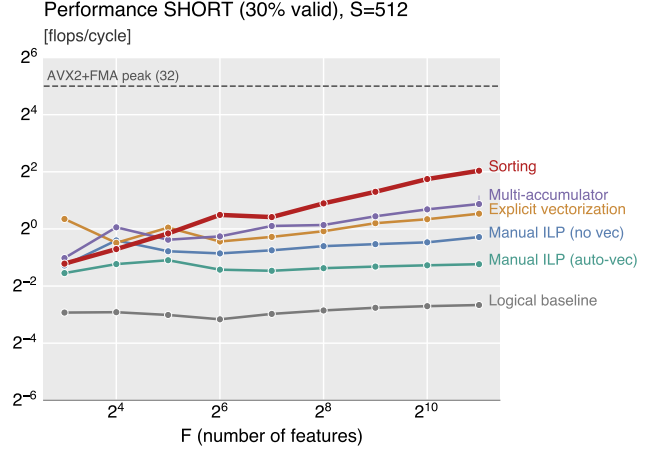
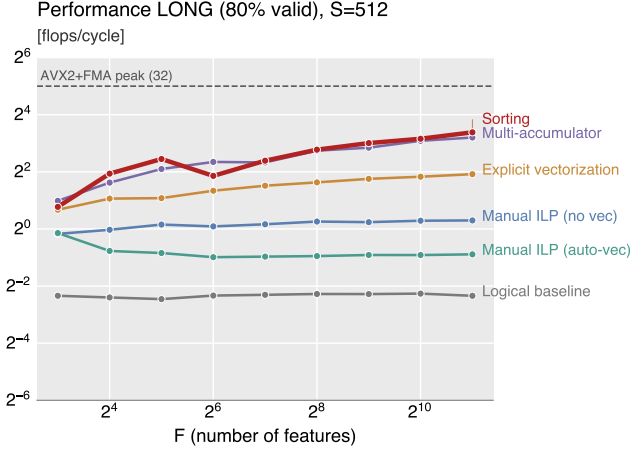


Fig. 2. Effective performance [flops/cycle] versus the number of features F at $S = 512$ in the LONG (left) and SHORT (right). Both axes are $\log_2$.

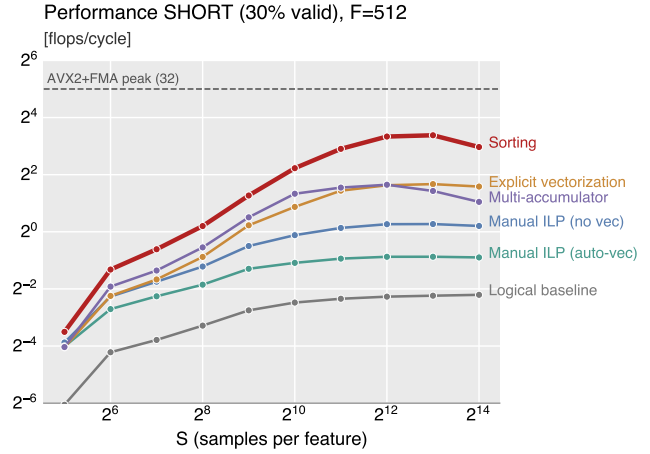
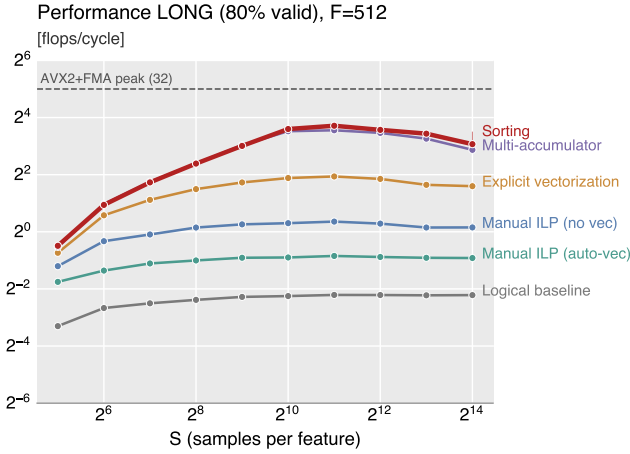


Fig. 3. Effective performance [flops/cycle] versus the number of samples per feature S at $F = 512$ in the LONG (left) and SHORT (right). Both axes are $\log_2$.

work done for one pair of features splits into two parts. The first is the dot product itself, whose length grows with S , so the longer the overlap, the more multiply-adds we perform. The second is a fixed amount of aggregation, i.e. summing the eight lanes of the vector accumulator into a single number, doing one division, and pushing the result into the top- K heap of each of the two features. This tail costs the same handful of instructions regardless of S , because it does not depend on how long the dot product was. That split is what shapes the curve. When S is small the dot product is short, so the constant tail accounts for a large share of the per-pair time and most cycles go to the second part rather than to FMAs. This is why performance starts low. As S grows the dot product lengthens while the tail stays fixed, so the same fixed cost is amortized over more

and more useful work and its relative weight shrinks. This is why every curve climbs steeply at first. The climb does not continue forever, because a second effect eventually takes over. The best LONG variant peaks at 13.1 flops/cycle (41% of peak) at $S = 2048$, the largest size at which the whole normalized matrix ($F \cdot S \cdot 4 = 4$ MB) still fits inside the 6 MB L3 cache and is therefore served from cache. Past that point the matrix no longer fits (8 MB at $S = 4096$, 32 MB at $S = 16384$) and has to be streamed from main memory on every pass, so performance falls back to 8.4 flops/cycle as DRAM bandwidth, rather than the tail, becomes the limit. SHORT peaks later, at $S = 8192$, because its overlaps are shorter. At the same S its dot products contain fewer valid samples, so it needs a larger S before they grow long enough to hide the same fixed tail, as explained before. This means

Table 1. PAPI hardware counters for each step at $F = 2048$, $S = 512$, medians over the timed batches. `perf` is analytical flops per cycle, `IPC` is instructions per cycle, and `ins/flop` is the instruction per real flop count not the analytical. `L*miss` is the different cache level misses and `%stl` is the fraction of cycles the core was stalled.

Step	LONG (80% valid)						SHORT (30% valid)					
	perf	IPC	ins/flop	L2 miss	L3 miss	%stl	perf	IPC	ins/flop	L2 miss	L3 miss	%stl
1 Logical baseline	0.20	2.71	13.8	131M	0.67M	16	0.16	2.47	15.7	31M	0.17M	17
2 Manual ILP (no vec)	1.23	3.32	2.7	128M	1.27M	2	0.82	2.55	3.1	31M	0.18M	8
3 Manual ILP (auto-vec)	0.54	1.65	3.1	130M	1.36M	32	0.42	1.66	3.9	28M	0.20M	22
4 Explicit vectorization	3.78	1.93	0.51	111M	1.37M	31	1.45	1.41	0.98	29M	0.17M	11
5 Multi-accumulator	9.19	2.28	0.25	27M	0.25M	27	1.83	2.12	1.16	16M	0.18M	24
6 Sorting	10.43	2.25	0.22	21M	0.39M	29	4.11	1.86	0.45	4.3M	0.14M	23

that once the algorithm is fast enough to be limited by computation, the bottleneck is on the constant work part 2 and not the dot product.

Roofline analysis. The performance plots tell us how fast we are. The roofline in Figure 4 tells us whether we are limited by computation or by memory. We compute operational intensity (OI) as the analytical flop count divided by the algorithm’s compulsory memory traffic, and the y -axis reuses the same cold-cache cycle counts as before. We do this as before to be able to compare each algorithm to the others. The ridge point, where the memory roof meets the compute roof, sits at ≈ 4.6 flops/byte. We measure the memory bandwidth of 7 B/cycle from a single-core STREAM-triad measurement of 19.8 GB/s at 2.8 GHz. Reading the plot left to right, at small sizes start under the memory roof: at $F = 8$ the OI is only 1.3 in `LONG` and 0.76 in `SHORT`, below the ridge, so there the algorithm is memory-bound and no amount of vector tuning would help. As F grows the dots march to the right because each loaded row is reused across more pairs. The useful flops grow like F^2 while the compulsory memory access grows only like F , so OI climbs to 220 in `LONG` and 76 in `SHORT` and the whole working point moves into the compute-bound roof. This is the key takeaway from the roofline. At the sizes we care about the algorithm has enough reuse that DRAM bandwidth is not the limit, so it is legitimate to keep optimizing the computation. The best `LONG` point lands at 10.4 flops/cycle, 33% of the AVX2+FMA roof. The S -sweep roofline in Figure 5 tells the same story from the other direction. Longer dot products raise OI and push the best variant up toward the roof, more steeply in `LONG` than in `SHORT`.

Limitations. Being limited by computation but still at only a third of peak means the bottleneck is now inside our algorithm implementation rather than in the memory system. We run a per-phase microbenchmark of the best variant and we saw that phase 2, the pairwise correlation loop, takes 97.8% of the cycles in `LONG` and 94.8% in `SHORT`, while the normalization (Phase 1) and the heap read-out (Phase 3) together stay under 5% and the sort itself under half a percent. Almost all of that Phase-2 time is the inner pair

loop, so this is the only place where remaining headroom can be found. Within that loop, we saw that the division is the most expensive piece. It is a scalar single-precision divide done once for every feature pair inside the Phase-2 loop nest (the $\text{acc}/(N - 1)$ that turns each finished dot product into a correlation), and on this core a division is a long-latency operation that the hardware does not pipeline the way it overlaps the multiply-adds. Thus we cannot hide the latency. The important point is that we only do this division once per pair of features no matter how long the overlap is. We also checked that the zero padding we added is not the bottleneck. We saw that after sorting the algorithm does only about 8% more floating-point work than strictly necessary. In short, the two FMA units are already saturated by the dot product, so the ceiling is set by this serial, scalar tail that we run once per pair and cannot widen to vectors or hide behind the multiply-adds.

5. CONCLUSIONS

Starting from a logical baseline that already exploited symmetry and a numerically stable variance formulation, we took the Interval-Masked Top- K Pearson correlation algorithm through different optimization and reached 10.4 flops/cycle, 33% of the AVX2+FMA peak, for a $53\times$ speedup on dense intervals and $26\times$ on sparse ones. Two findings stand out. First, the masked inner loop did not auto-vectorize usefully, that is, letting the compiler vectorize it was actually slower than clean scalar code, and the real gains came only once we vectorized by hand on top of a zero-padding scheme that made the masked dot product dense and branch-free. Second, the value of an optimization depended heavily on the input data, with interval sorting almost irrelevant on dense data but the single most important step on sparse data, where its early exit removed the wasted work the padding introduced. The roofline showed that at the problem sizes of interest the kernel is limited by computation rather than by memory bandwidth, and a per-phase microbenchmark localized the remaining gap to peak inside the Phase-2 pair loop, which alone accounts for over 94% of the runtime: not

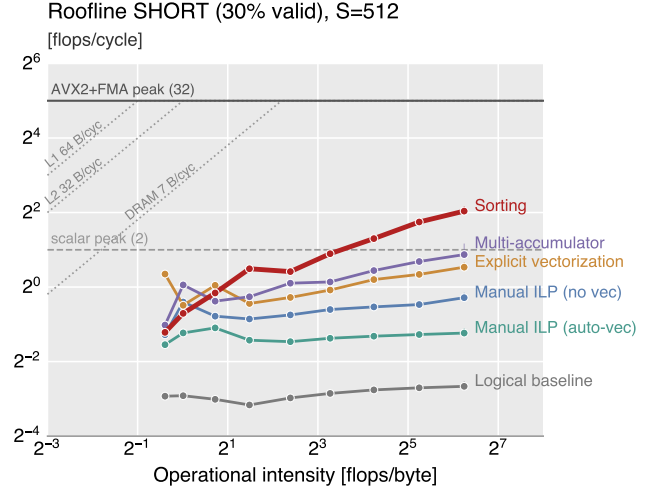
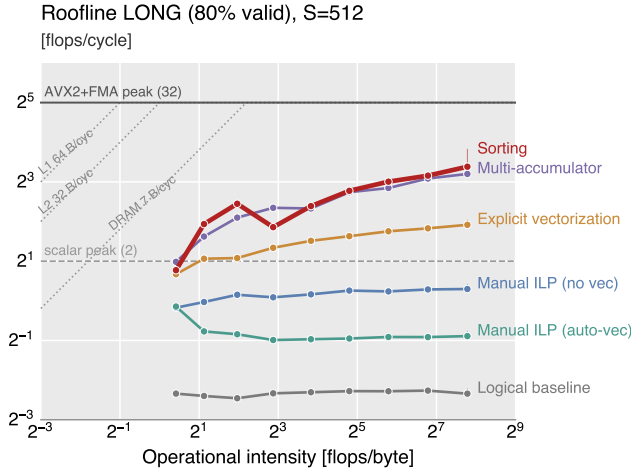


Fig. 4. Roofline for the LONG (left) and SHORT (right). Each dot is one F along the sweep ($S = 512$), connected in order. Operational intensity is the analytical flop-to-compulsory-byte ratio of the cost model.

the vectorized dot product, which already keeps both FMA units busy. The main bottleneck still is on the fixed scalar work done once per feature pair, dominated by the long-latency division that normalizes each finished dot product.

6. REFERENCES

- [1] Karl Pearson, “Note on regression and inheritance in the case of two parents,” *Proc. Royal Soc. London*, vol. 58, pp. 240–242, 1895.
- [2] Peter Langfelder and Steve Horvath, “Wgcna: an R package for weighted correlation network analysis,” *BMC bioinformatics*, vol. 9, no. 1, pp. 559, 2008.
- [3] Spiros Papadimitriou, Jimeng Sun, and S Yu Philip, “Local correlation tracking in time series,” in *Sixth International Conference on Data Mining (ICDM’06)*. IEEE, 2006, pp. 456–465.
- [4] Erich Schubert and Michael Gertz, “Numerically stable parallel computation of (co-)variance,” in *Proc. 30th Int. Conf. on Scientific and Statistical Database Management (SSDBM)*. ACM, 2018, pp. 10:1–10:12.
- [5] Hui Xiong, Shashi Shekhar, Pang-Ning Tan, and Vipin Kumar, “TOP-COP: Mining top- k strongly correlated pairs in large databases,” in *Proc. 6th IEEE Int. Conf. on Data Mining (ICDM)*. IEEE, 2006, pp. 1162–1166.
- [6] Innovative Computing Laboratory, University of Tennessee, “PAPI: The performance application programming interface,” <https://github.com/icl-utk-edu/papi>, Accessed 2026.

7. CONTRIBUTIONS OF TEAM MEMBERS (MANDATORY)

Luhao. Implemented the explicit AVX2/FMA vectorization of the Phase-2 dot product (step 4) and the Phase-3 min-heap. Came up with the “logical” optimizations in the logical baseline (step 1).

Andres. Wrote and implemented the decomposed incremental stages (the manual-ILP variants with and without auto-vectorization, steps 2 and 3, and the interval-sorting step, step 6). Improved the vectorization solution to exploit full hardware. Study the performance of each algorithm to understand the bottlenecks.

Oliver. Performance and roofline analysis across all variants plus the plotting, analyzing the isolated effect of each optimization stage. Came up with the interval-sorting idea.

8. CODE (MANDATORY)

This section does not count towards the 7 pages limit.

Please copy-paste the most performance-critical part of your fastest scalar code here. Should be at most one page. If you have more than one such part, choose one. Could be done in single column if need be. For best display consider adding a page break in latex before.

Paste in here, do not change font size:

```
for( size_t i = 0; i < N; i+=2 )
for( size_t j = 0; j < N; j+=2 )
for( size_t k = 0; k < N; k+=2 ) {

    double t0, t1, t2, t3, t4, t5, t6, t7,
           t8, t9, t10, t11, t12;
```

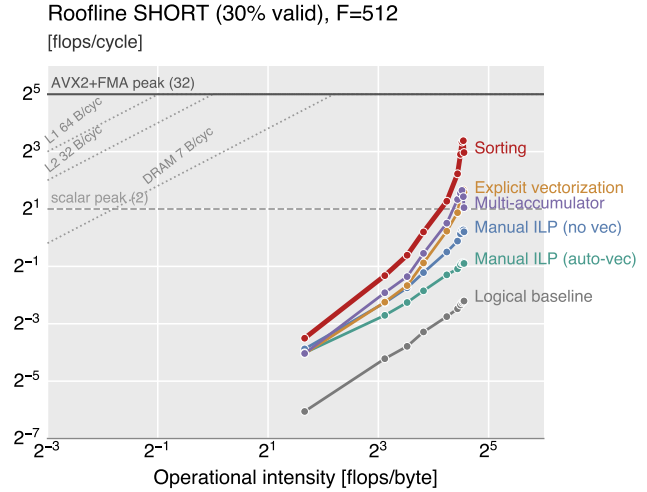
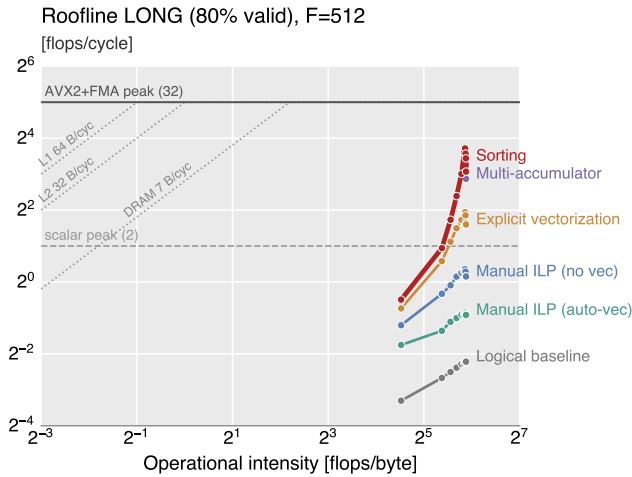



Fig. 5. Roofline for the LONG (left) and SHORT (right) when sweeping $S \in \{32, \dots, 16384\}$ at $F = 512$; each dot is one S , connected in order. Larger S raises operational intensity and moves the best variant up toward AVX2+FMA peak.

```

t7 = A[N*i + k];
t6 = A[N*i + k + 1];
t5 = A[N*i + N + k];
t4 = A[N*i + N + k + 1];
t3 = B[j + N*k];
t2 = B[j + N*k + 1];
t1 = B[j + N*k + N];
t0 = B[j + N*k + N + 1];
t8 = C[N*i + j];
t9 = C[N*i + j + 1];
t10 = C[N*i + N + j];
t11 = C[N*i + N + j + 1];
t12 = t7 * t3;
t8 = t8 + t12;
t12 = t7 * t2;
t9 = t9 + t12;
t12 = t5 * t3;
t10 = t10 + t12;
t12 = t5 * t2;
t11 = t11 + t12;
t12 = t6 * t1;
t8 = t8 + t12;
t12 = t6 * t0;
t9 = t9 + t12;
t12 = t4 * t1;
t10 = t10 + t12;
t12 = t4 * t0;
t11 = t11 + t12;
C[N*i + j] = t8;
C[N*i + j + 1] = t9;
C[N*i + N + j] = t10;
C[N*i + N + j + 1] = t11;

```

After that, please paste the most performance-critical part of your fastest SIMD code here. Again, this should be at most one page. If you have more than one such part, choose one. Could be done in single column if need be. For

best display consider adding a page break in latex before.
Paste in here, do not change font size:

```

ones = _mm256_set1_pd(1.);
mones = _mm256_set1_pd(-1.);
thresholds = _mm256_set1_pd(0.5);
for(i = 0; i < n; i+=4) {
    vt = _mm256_load_pd(x+i);
    vmask = _mm256_cmp_pd(vt, thresholds, _CMP_GT_OQ);
    vp = _mm256_and_pd(vmask, ones);
    vm = _mm256_andnot_pd(vmask, mones);
    vr = _mm256_add_pd(vt, _mm256_or_pd(vp, vm));
    _mm256_store_pd(x+i, vr);
}

```